

Configure apps on Azure Kubernetes Service

In this exercise, you learn how to configure Kubernetes deployments with ConfigMaps for non-sensitive settings, Secrets for sensitive credentials, and PersistentVolumeClaims for persistent storage. You deploy a containerized API to Azure Kubernetes Service (AKS), configure it with various Kubernetes resources, and interact with it using a Python client application.

Tasks performed in this exercise:

- Download the project starter files
- Deploy resources to Azure (ACR, AKS cluster)
- Build and push a container image to Azure Container Registry
- Configure kubectl credentials for AKS cluster access
- Apply updated YAML files to AKS to create the pod and expose the API with a LoadBalancer
- Run the client app to test the API endpoints
- View API logs stored on persistent volume
- Clean up Azure resources

This exercise takes approximately **30** minutes to complete.

IMPORTANT: The persistent storage implementation in this exercise is for demonstration purposes only. For logging, production applications should use a centralized logging solution like Azure Monitor or Application Insights instead of storing logs on persistent volumes. If persistent storage is required, implement log rotation policies to prevent storage from filling up, which can cause container failures and pod evictions.

Important: Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- The Kubernetes command-line tool, [kubectl](#).
- [Python 3.12](#) or greater.

Download project starter files and deploy Azure services

In this section you download the starter files for the console app and use a script to deploy the necessary services to your Azure subscription. The Azure resource deployments can take 10-15 minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

Code

Copy

```
https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aks-configure-python.zip
```

2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.

3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
4. Open the *azdeploy.py* deployment script and change the resource group and location values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

CodeCopy

```
rg = "<your-resource-group-name>" # Resource Group name
location = "<your-azure-region>" # Azure region for the resources
```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

CodeCopy

```
az login
```

7. Run the following commands to ensure your subscription has the necessary resource providers to install AKS and ACR. The **Microsoft.Compute**, **Microsoft.Network**, and **Microsoft.Storage** providers are dependencies of AKS that Azure usually registers automatically, but explicitly registering them avoids occasional cluster creation failures on new subscriptions.

Before you start

[Download project starter files and deploy Azure services](#)

[Complete the YAML deployment files and deploy to AKS](#)

[Apply the manifests to AKS](#)

[Run the client app](#)

[Clean up resources](#)

[Troubleshooting](#)

CodeCopy

```
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Compute
az provider register --namespace Microsoft.Network
az provider register --namespace Microsoft.Storage
```

8. Make sure you are in the root directory of the project and run the following command to launch the deployment script.

CodeCopy

```
python azdeploy.py
```

 **Note:** Depending on your system configuration, the Python command might be **python3** instead of **python**.

Deploy resources to Azure

With the deployment script running, follow these steps to create the needed resources in Azure.

1. Enter **1** to launch **Create Azure Container Registry (ACR)**. This creates the resource where the API container will be stored, and later pulled into the AKS resource.
- When the operation is complete it will return the ACR endpoint. Copy the information, you need it later in the exercise.
2. After the ACR resource has been created, enter **2** to launch **Build and push API image to ACR**. This option uses ACR tasks to build the image and add it to the ACR repository. This operation can take 3-5 minutes to complete.

- After the image has been built and pushed to ACR, enter **3** to launch the **Create AKS cluster** option. This creates the AKS resource configured with a managed identity, gives the service permission to pull images from the ACR resource, and assigns the needed RBAC role to write to the persistent storage. This operation can take 5-10 minutes to complete.
- After the AKS cluster deployment has completed, enter **4** to launch the **Get AKS credentials for kubectl** option. This uses the **az aks get-credentials** command to retrieve credentials and configure **kubectl**.
- After the credentials have been configured, enter **5** to launch the **Check deployment status** option. This option reports if each of the resources have been successfully deployed.

If all of the services return a **successful** message, enter **7** to exit the deployment script.

If the AKS cluster is in a **Failed** or **Canceled** state, correct the reported issue, then enter **6** to launch the **Delete failed AKS deployment** option before running option **3** again. This guarded option does not delete a healthy or in-progress cluster.

Next, you complete the YAML files necessary to deploy the API to AKS.

Complete the YAML deployment files and deploy to AKS

In this section you complete YAML files, located in the *k8s* folder, needed to configure Kubernetes deployments with persistent storage, and store sensitive and non-sensitive settings.

Complete the ConfigMap YAML file

ConfigMaps store non-sensitive configuration data as key-value pairs that can be consumed by pods. In this section, you create a ConfigMap to store application settings like the student name, API version, and log path.

- Open the *k8s/configmap.yaml* file and add the following code to the file. You can update the value for **STUDENT_NAME** with your name if you want to.

CodeCopy

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  labels:
    app: aks-config-api # Label for the AKS configuration API
data:
  # Store non-sensitive configuration values
  STUDENT_NAME: "YourNameHere"
  API_VERSION: "1.0.0"
  LOG_PATH: "/var/log/api" # Path for API logs
```

- Review the comments in the code, then save your changes.

Complete the Secrets YAML file

Secrets store sensitive information like passwords, tokens, and keys in a base64-encoded format. In this section, you create a Secret to store sensitive credentials that the API will access at runtime.

- Open the *k8s/secrets.yaml* file and add the following code to the file.

CodeCopy

```
apiVersion: v1
kind: Secret
metadata:
  name: api-secrets
  labels:
    app: aks-config-api
type: Opaque
stringData:
  # Store sensitive credentials as base64-encoded values
  secret-endpoint: "SecretEndpointValue"
  secret-access-key: "SecretAccessKey123456"
```

2. Take a few minutes to review the comments in the code, then save your changes.

Complete the PVC YAML file

A PersistentVolumeClaim (PVC) requests storage resources from Azure that can be mounted to pods. In this section, you create a PVC that uses Azure Disk storage to persist API log files across pod restarts.

1. Open the *k8s/pvc.yaml* file and add the following code to the file.

CodeCopy

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: api-logs-pvc
  labels:
    app: aks-config-api # Label for the AKS configuration API
spec:
  accessModes:
    - ReadWriteOnce # Allow single pod to mount volume for read/write
  resources:
    requests:
      storage: 1Gi # Request minimum Azure Disk size
  storageClassName: managed-csi # Use Azure Disk CSI driver (default)
  volumeMode: Filesystem # Default mode
```

2. Take a few minutes to review the comments in the code, then save your changes.

Update the Deployment YAML file

The deployment manifest is already partially configured with environment variables, volume mounts, and probes. You just need to update the container image reference with your specific ACR endpoint.

1. Open the *k8s/deployment.yaml* file and locate the **image: <YOUR_ACR_ENDPOINT>/aks-config-api:latest** line.
2. Replace **<YOUR_ACR_ENDPOINT>** with the value you recorded earlier in the exercise.
3. Take a few minutes to review the comments in the code, then save your changes.

Apply the manifests to AKS

In this section you apply the manifests to AKS. The following steps are performed in the VS Code terminal. Ensure you are in the root of the project before running the commands.

1. Run the following command to apply the ConfigMap.

Code	 Copy
<pre>kubectl apply -f k8s/configmap.yaml</pre>	

2. Run the following command to apply the Secrets.

Code	 Copy
<pre>kubectl apply -f k8s/secrets.yaml</pre>	

3. Run the following command to apply the PersistentVolumeClaim.

Code	 Copy
<pre>kubectl apply -f k8s/pvc.yaml</pre>	

4. Run the following command to apply the Deployment.

Code	 Copy
<pre>kubectl apply -f k8s/deployment.yaml</pre>	

5. Run the following command to create the Service.

Code	 Copy
<pre>kubectl apply -f k8s/service.yaml</pre>	

6. After you create the Service it can take a few minutes for the deployment to complete. The following command will monitor the service and update the external IP address of the pod when it's available. Enter **ctrl + c** to exit the command after the IP address appears.

Code	 Copy
<pre>kubectl get svc aks-config-api-service -w</pre>	

7. Run the following command from the project root to launch the deployment script again.

Code	 Copy
<pre>python azdeploy.py</pre>	

8. Enter **5** to run the **Check deployment status** option. The script reads the LoadBalancer IP and writes it to *client/.env* as **API_ENDPOINT**. When the status check completes, enter **7** to exit the deployment script.

Run the client app

In this section, you configure the Python environment and then perform operations on the API using the client app

Configure the Python environment

In this section, you create the Python environment and install the dependencies.

1. Ensure you are in the *client* folder of the project in the terminal.
2. Run the following command in the VS Code terminal to create the Python environment.

CodeCopy

```
python -m venv .venv
```

3. Run the following command to activate the Python environment. **Note:** On Linux/macOS, use the Bash command. On Windows, use the PowerShell command. If using Git Bash on Windows, use **source .venv/Scripts/activate**.

Bash

CodeCopy

```
source .venv/bin/activate
```

PowerShell

CodeCopy

```
.\.venv\Scripts\Activate.ps1
```

4. Run the following command in the VS Code terminal to install the dependencies.

CodeCopy

```
pip install -r requirements.txt
```

Perform operations with the app

With the Python environment configured and dependencies installed, you can now run the client application to test the deployed API. The API logs all operations to the persistent volume, and the client provides a menu-driven interface to interact with various endpoints.

1. Run the following command in the terminal to start the console app. Refer to the commands from earlier in the exercise to activate the environment, if needed, before running the command.

CodeCopy

```
python main.py
```

2. Enter **1** to start the **Check API Health (Liveness)** option. This verifies that the API container is running and responds to health checks, which is the same endpoint used by the Kubernetes liveness probe. Note the information it returns contains the non-sensitive student name set in ConfigMap.

CodeCopy

```
[*] Checking API health...
✓ API is healthy
Service: aks-config-api
Version: 1.0.0
Student: YourNameHere
```

3. Enter **2** to start the **Check API Readiness (Foundry Connectivity)** option. This confirms the API can successfully connect to the Foundry model endpoint and is ready to process inference requests.

4. Enter **3** to start the **View Secrets Information** option. This functionality exists only so you can confirm your secrets were set in the pod and is for demonstration purposes only. In the output you can view information about the secrets, the output is masked.

CodeCopy

```
Secret Details:

secret_endpoint:
  Loaded: True
  Value: SecretEndp...
  Length: 19 characters

secret_access_key:
  Loaded: True
  Value: ***3456
  Length: 21 characters
```

5. Enter **5** to start the **List All Products** option. This displays the mock data included in the API.
6. Now that the API has logged operations for several different endpoints, it's time to view the logs. Enter **6** to start the **View Log Summary** option and see a summary of the different operations. Note the total number of requests, and the requests to the **/readyz** and **/healthz** endpoints. Those two operations are executing automatically based on the schedule set in the *deployment.yaml* file.

CodeCopy

```
✓ Log summary retrieved

Log file: /var/log/api/api-requests-2025-12-21.log
Total requests: 220
Student: YourNameHere

First request: 2025-12-21T02:19:48.953308
Last request: 2025-12-21T02:46:08.952772

Requests by endpoint:
  /readyz: 160
  /healthz: 56
  /secrets: 1
  /products: 1
```

7. You can continue to generate log information and when you're finished enter **7** to exit the app.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

CodeCopy

```
az group delete --name <rg-name> --no-wait --yes
```


CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues while completing this exercise, try the following troubleshooting steps:

Check deployment status with the deployment script

- Run the deployment script and select option **5. Check deployment status** to verify the state of all deployed resources.
- This command checks:
 - ACR provisioning state and readiness
 - AKS cluster provisioning state
 - Kubernetes resources (ConfigMap, Secrets, PVC, Deployment availability, Service LoadBalancer IP)
- Use this output to identify which component may be causing issues.

Resolve AKS cluster creation failures

- Quota validation can fail before an AKS resource is created, while failures later in provisioning can leave the cluster in a **Failed** or **Canceled** state.
- If the error reports that **Standard_D2s_v7** is unavailable or that the region has insufficient capacity, exit with option **7**. Change **AKS_VM_SIZE** to one of the v5 or v6 fallback sizes listed near the top of the deployment script, or change the **location** value, and then run option **3. Create AKS cluster** again.
- If the error reports insufficient quota, select a region where your subscription has available Dsv5-family quota or request a quota increase. Changing regions only helps when the other region has sufficient quota.
- The AKS cluster uses the **location** configured in the script even when the resource group already exists in another region.
- If option **5** reports **Failed** or **Canceled**, correct the underlying issue and run option **6. Delete failed AKS deployment** before retrying option **3**. If no AKS resource was created, no deletion is needed.

Verify YAML file completeness

- Ensure all YAML content was added correctly to *configmap.yaml*, *secrets.yaml*, and *pvc.yaml* and that indentation is correct.
- Confirm the ACR endpoint was properly updated in *deployment.yaml* (replace **<YOUR_ACR_ENDPOINT>** with your actual ACR endpoint).
- If you make changes to any YAML file after initial deployment, reapply the file with **kubectl apply -f k8s/<filename>.yaml**.
- After updating ConfigMap or Secret files, perform a rolling restart to reload the configuration: **kubectl rollout restart deployment aks-config-api**.

Verify client configuration

- Check that *client/.env* exists and contains **API_ENDPOINT** set to the LoadBalancer's external IP (for example, <http://20.xxx.xxx.xxx>).
- If the file is missing or shows a stale IP, run the deployment script and select option **5. Check deployment status** after the Service has an external IP to rewrite *client/.env*.
- Verify you can reach the API endpoint by running **curl http://<external-ip>/healthz** from the terminal.

Check Python environment and dependencies

- Confirm the virtual environment is activated before running the client app.
- Verify that all packages from *requirements.txt* were installed successfully by running **pip list**.
- Ensure you're running the client from the *client* directory.

